

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа № 3

Выполнил:

Лесовой Никита

Группа

J3214

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Мигрировать ранее разработанное приложение на фреймворк Vue.js. Для варианта 3 требовалось сохранить работу с JSON Server и Open-Meteo, добавить навигацию без полной перезагрузки, общее состояние Pinia, защищённые страницы и повторно используемые компоненты.

Ход работы

1. Создание Vue-проекта. В папке lab3 настроен Vite, подключены Vue 3, Vue Router, Pinia, pinia-plugin-persistedstate, Axios, Bootstrap и JSON Server. Точка входа src/main.js создаёт приложение, подключает маршрутизатор, хранилища и общий файл стилей, после чего монтирует App.vue.

2. Разделение на слои. Работа с ресурсами вынесена в src/api: отдельные модули отвечают за вход, направления, поездки и погоду. Каталог, авторизация и поездка хранятся в src/stores. Переиспользуемые части интерфейса размещены в components, общая навигация — в layouts/BaseLayout.vue, а страницы приложения — в views.

3. Pinia и сохранение состояния. Хранилище auth содержит currentUser и token, store destinations отвечает за каталог, поиск, фильтры, загрузку данных и сохранение маршрута, а store trip — за поездку, заметки и процент бюджета. Для безопасной работы с реактивностью используются storeToRefs и действия store. Плагин persistedstate сохраняет выбранные поля авторизации между перезагрузками.

Листинг 1 — вычисляемая фильтрация в Pinia-хранилище

```
const filteredDestinations = computed(() => {
  const searchValue = filters.search.trim().toLowerCase();

  return destinations.value.filter((destination) => {
    const matchesSearch = destination.name.toLowerCase().includes(searchValue);
    const matchesType = filters.type === "all" || destination.type === filters.type;
    const matchesBudget = filters.budget === "all" || destination.budget === filters.budget;
    const matchesDuration = filters.duration === "all" || destination.duration === filters.duration;

    return matchesSearch && matchesType && matchesBudget && matchesDuration;
  });
});
```

4. Маршрутизация и компоненты. В router/index.js описаны страницы каталога, направления, входа, регистрации, профиля, поездки и страницы 404. Динамический путь /destination/:id передаёт идентификатор направления, а navigation guard перенаправляет неавторизованного пользователя на /login. Компоненты DestinationFilters, DestinationList, DestinationCard и WeatherPanel получают данные через props и события.

Листинг 2 — защита маршрутов в router/index.js

```
router.beforeEach((to) => {
  const authStore = useAuthStore(pinia);

  if (to.meta.requiresAuth && !authStore.isAuthenticated) {
    return { name: "login", query: { redirect: to.fullPath } };
  }

  if (to.meta.guestOnly && authStore.isAuthenticated) {
    return { name: "profile" };
  }

  return true;
});
```

5. Представления и composable. DestinationsView загружает каталог и показывает поиск, DestinationView отображает подробности направления и прогноз, ProfileView показывает сохранённые маршруты, а TripView управляет бюджетом, участниками и заметками. Функции useFormatters используются повторно для форматирования цены, дат, длительности и погодных значений. Благодаря этому шаблоны остаются компактными.

Листинг 3 — динамическая ссылка в DestinationCard.vue

```
<div class="d-flex justify-content-between align-items-center gap-3 mt-auto">
  <strong>от {{ formatPrice(destination.price) }}</strong>
  <RouterLink class="btn btn-primary btn-sm" :to="`~/destination/${destination.id}`">
    Подробнее
  </RouterLink>
</div>
```

6. Запуск и тестирование. В первом окне PowerShell запускается npm.cmd run api, во втором — npm.cmd run dev. После открытия адреса Vite проверены загрузка каталога, поиск, динамическая страница направления, guard авторизации, профиль, план поездки и удаление заметки. Дополнительно выполнена команда npm.cmd run build, чтобы проверить успешность production-сборки.

Вывод

Итоговая лабораторная работа превратила многостраничный статический проект в одностраничное приложение на Vue 3. Router отвечает за смену представлений, Pinia объединяет состояние, Axios инкапсулирует API-запросы, а компоненты и composables сокращают повторение кода. Приложение сохраняет функциональность предыдущих этапов и при этом получает более понятную структуру, реактивное обновление интерфейса и защиту маршрутов.